

Project: Managing Multiple instances of a service.

Done by: Aashish Chaudhary (04/28/2026)

Concepts: Working with Service files, pid file, and managing services

Used Service: Nginx.

Project Overview:

In this project, I have setup multiple instances of a nginx service, using a single service file. The core idea is, in production environment, the engineers might have to configure multiple virtual hosts, completely separate that do not stomp on each other. To simulate this, I have configured these two instances on separate ports (6060 AND 9090) of a single machine. To test if nginx serves the webpages, I have two sites (Site1 AND Site2) for each services. Using this documentation, we can setup as many instances, as the core idea of setting up remains same. The approach of accomplishing tasks remains more, as it depends on the person doing it. My attempt is having separate configs for each nginx instances. For that, I have used two nginx.conf files, each for one individual service.

How does a NGINX generally serve?

User ----> requests url ---->Nginx ---->gets and serves webpages.

To Serve Webpages, we need Nginx. But how does a service nginx is created? Here's how its created:

Computer ----->systemd----->reads /etc/systemd/system/nginx.service file

Ok now, what is this nginx.service file?

This file has the information about the service we are trying to create. For instance, if I am creating nginx service, I will add the things like, "what it is?", " when to create, what to do after creating it", "what to do if the service fails?" and other things like this. We also add which config files to read in it. The systemd uses this information and creates a service accordingly.

BOOM!!!! That's the whole process. If it is clear up to this point, rest is easy as a piece of cake. The rest is finding configs on google, using commands, and working through files. There are couple of things to remember. I will explain it on the next section.

Ok, now, we know, Systemd creates a service using information on `/etc/systemd/system/service.file`. In this service file, we add line to execute command that reads or reloads, which nginx configuration file we want the **systemd** to use while creating it. So, important thing is writing `nginx.conf` files for each services. So,

Systemd -----reads config -----from Service file (i.e. has `nginx.conf` info, pid info)

Now, for any nginx service, it needs some webpage to host, some place to store its logs, and some place to store its `.pid` file. And the owner of these directory must be “www-user”. Do not forget to create them.

Think of this, a restaurant (Nginx) serves cooked food (webpage), needs pantry(logs place) to store food item, and a dumpster to throw trash (error_log place). And both should be owned by Restaurant.

The Nginx Config file has all the information about who the user is, what files to serve, where to log access_logs, error_logs, what port to listen on. Things like these let the newly created service how to behave. So, here's the final diagram that clicks everything together.

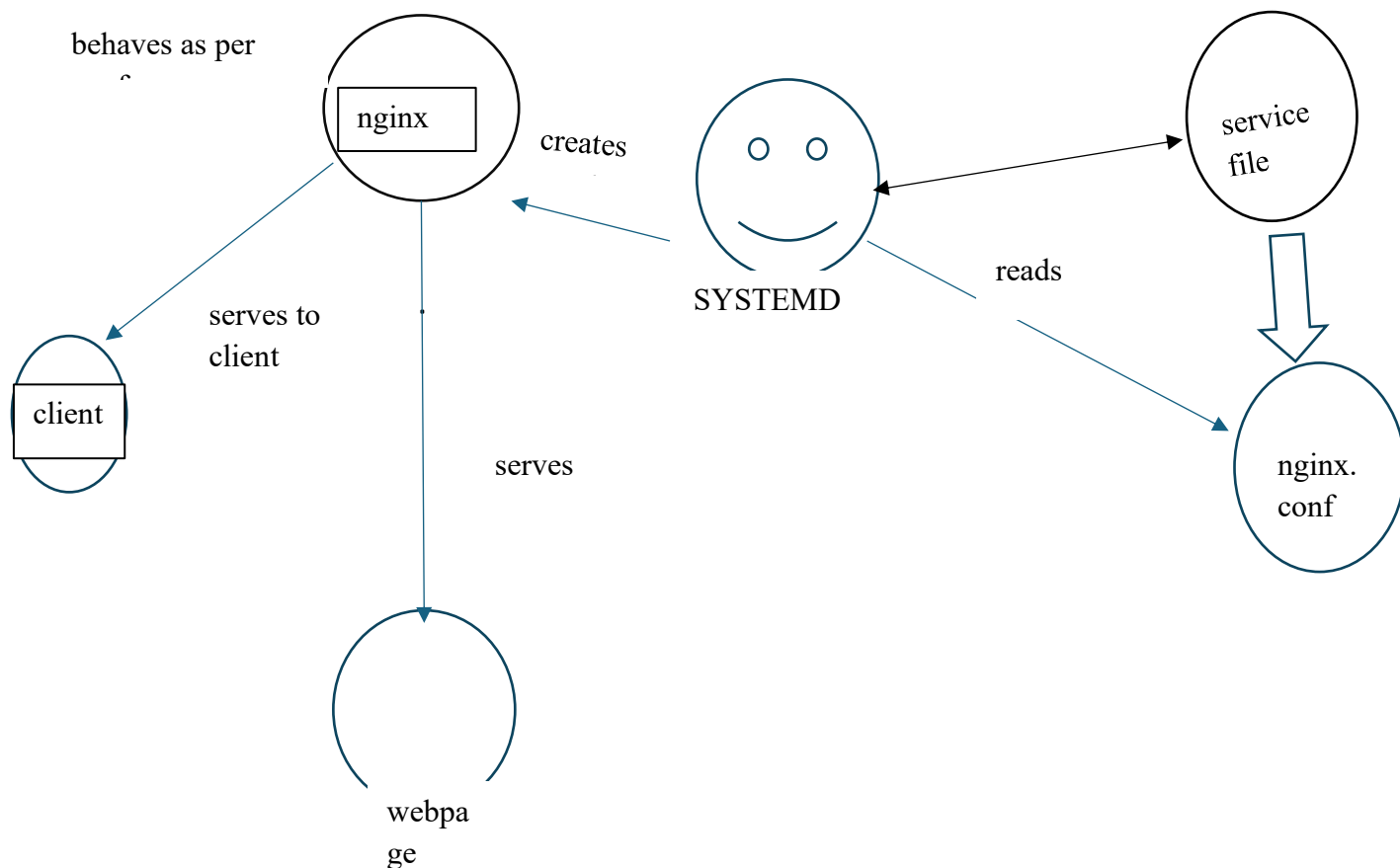


fig: Architecture of Systemd creating nginx.

This is how nginx service is created. Now, we are creating two of the instances. So i used %i in my service file. This way, the systemd uses argument passed to tcommand after @. And the service runs fine.

For this project, since I am creating two services, I have created following paths for files and logs. Both services have these things in common. I have included only one in here.

Service 1:

Config: /etc/nginx-site1/nginx.conf

Log: /var/log/nginx-site1/

Pid: /run/nginx-site1/

Site: /var/www/site1

Note: Except for config file, Rest directories are owned by user “www-user”.

Finally, systemd commands will start the service and troubleshoot if any issue persists using **sudo systemctl** or **journalctl** commands. In this way, I have attempted to manage multiple instances of a service. I have successfully troubleshooted the issues that I encountered while doing this project.